

ISO/TC 184/SC 5/WG 17

“Interoperability of simulation models on different manufacturing platforms”

ISO 21175-2

Automation systems and integration —
Collaboration Environment Requirements of Simulation
on Different Manufacturing Platforms

Part 2: Message Meta-model and Exchange Formats for Collaborative Distributed Simulation

DRAFT

Date: June 4, 2026

This draft document specifies the message meta-model and exchange formats supporting the collaborative modelling and simulation environment (CMSE) introduced in ISO 21175-1.

Contents

Introduction	3
1 Scope	5
2 Normative references	5
3 Terms and definitions	6
4 Abbreviated terms	6
5 Conformity with this part of ISO 21175	7
5.1 General conformance requirements	7
5.2 Conformance classes	8
5.2.1 Coordinator conformance class	8
5.2.2 Runner conformance class	8
5.2.3 Monitoring conformance class	8
5.3 Mandatory and optional features	9
5.4 Statement of conformity	9
5.5 Conformance testing	9
6 General framework of collaborative message exchange	10
6.1 Roles in message-based collaborative simulation	11
6.2 Logical communication topology	11
6.3 Relation to business and system description	12
6.4 Simulation lifecycle and message flow	12
6.5 Independence from infrastructure deployment	14
7 Message meta-model	14
7.1 Core concepts and structural elements	14
7.2 Identification and addressing	16
7.3 Time representation	16
7.4 State and payload description	17
7.5 Relationship to business and system description	18
8 Standard message sets for collaborative simulation	18
8.1 Simulation lifecycle management messages	19
8.1.1 SimulationInit	19
8.1.2 SimulationTerminate	20
8.1.3 ModelTerminated	21
8.1.4 Ordering constraints for lifecycle messages	21
8.2 Time management and scheduling messages	22
8.2.1 NextInternalTimeReport	22
8.2.2 RequestOutput	22
8.2.3 OutputReport	23
8.2.4 ExecuteTransition	23
8.2.5 TransitionComplete	24
8.2.6 Ordering constraints for time management messages	24
8.3 Model input and output messages	25
8.3.1 Port values in OutputReport messages	25
8.3.2 Port values in ExecuteTransition messages	25
8.3.3 Ordering constraints for model input and output	26

8.4	Monitoring, logging and result reporting messages	26
8.4.1	MonitoringMessage	26
8.4.2	Result reporting	27
8.4.3	Ordering constraints for monitoring and logging	27
8.5	Error handling and diagnostic messages	27
8.5.1	ErrorReport	27
8.5.2	Use of ErrorReport in the protocol	28
9	Serialization and transport bindings	28
9.1	JSON-based encoding	29
9.1.1	Common JSON structure	29
9.1.2	Encoding of primitive types and payloads	29
9.1.3	Encoding of logical time	30
9.1.4	Port-level payloads	30
9.1.5	Message-type specific constraints	31
9.1.6	Illustrative example	33
9.2	Transport protocol considerations	34
9.2.1	General principles	34
9.2.2	Shared logical channel organization	34
9.2.3	Delivery and implementation considerations	35
9.2.4	Ordering and partitions	35
9.2.5	Delivery guarantees and recovery	35
9.2.6	Security and isolation	36
10	Extensibility and future work	36
Annex A (informative)	Example message sequences for a simple collaborative simulation	36
Bibliography		40

Foreword

This document was prepared by Technical Committee ISO/TC 184, Automation systems and integration, Subcommittee SC 5, Interoperability of simulation models on different manufacturing platforms.

ISO 21175 consists of the following parts, under the general title “Collaboration Environment Requirements of Simulation on Different Manufacturing Platforms”:

- Part 1: Reference model and process
- Part 2: Message meta-model and exchange formats for collaborative distributed simulation
- Part 3: Model representation for exchange and reuse
- Part 4: API-based collaborative execution and model exchange

Part 1 establishes a reference model and reference process for a Collaborative Modelling and Simulation Environment (CMSE), providing a general framework to support joint simulation projects across heterogeneous manufacturing platforms.

Part 2 (this document) specifies an abstract message meta-model and a set of standardized message sets and exchange formats to support distributed and re-distributed simulation in a collaborative setting. It formalizes how simulation models exchange information, manage logical time and share results across different platforms and organizations.

Part 3 will specify a standardized representation of simulation models so that they can be identified, inspected and exchanged in a consistent way. It will define how a model is named, how its interface is described (including its ports and the types and roles of these ports), and which additional model metadata are required to support composition, discovery and automated processing. The representation is intended to be simple, technology-neutral and machine-readable, so that tools conforming to this part can, for example, list available models, expose their ports for drag-and-drop composition, and integrate imported models into simulation configurations without ambiguity.

Part 4 will specify how these models and their executions are exposed and orchestrated through service interfaces and application programming interfaces (APIs). It will define mechanisms for remote and automated execution of models, including starting, controlling and monitoring simulations without manual intervention, and will describe how models can be published, discovered and exchanged via APIs to support autonomous and scalable collaborative simulation workflows across organizations.

Introduction

Open, sharing and on-demand collaboration among enterprises or departments using simulation models throughout the product life cycle continues to grow in importance. To support such collaboration, simulation models must be able to interoperate across heterogeneous manufacturing platforms, software frameworks and infrastructures, while still preserving the semantics of their behaviour and of the data they exchange. In this context, the Collaborative Modelling and Simulation Environment (CMSE) defined in ISO 21175-1 provides a reference model and reference process for organizing joint simulation projects, but it does not yet specify in detail how messages used during simulation should be structured and exchanged.

In many current joint simulation projects, model interaction is still realized through ad-hoc or platform-specific interfaces. Simulation stakeholders often need to negotiate message formats informally, to integrate software and models offline, or even to start and control simulations manually. These practices limit reuse of models, make cross-platform collaboration costly, and hinder the deployment of large-scale distributed or re-distributed simulations involving partners

from different organizations. A standardized, technology-neutral message meta-model is therefore needed to ensure that simulation messages can be understood and processed consistently, independently of the underlying implementation technology or middleware.

This part of ISO 21175 specifies an abstract message meta-model and a set of exchange formats for distributed and re-distributed simulation in a collaborative environment. It refines the reference model and process defined in ISO 21175-1 by formalizing how simulation models communicate, synchronize logical time and share simulation results across heterogeneous platforms. The message meta-model describes the structure and semantics of simulation messages, including identifiers, time information, ports and payloads, while the associated message sets define standard patterns for simulation lifecycle management, time management, model input/output, monitoring and diagnostics.

Within the overall series of standards, this part is intended to complement the general CMSE framework of ISO 21175-1 and to provide a basis for the subsequent parts. Part 3 will specify how individual simulation models are represented, named and described (including their ports and other interface metadata) so that they can be inspected, composed and exchanged in a consistent and machine-readable way. Part 4 will build on the message meta-model and model representation to define service interfaces and application programming interfaces (APIs) for remote and automated execution and exchange of models, enabling collaborative simulation workflows without requiring manual operation of the underlying platforms.

1 Scope

This part of ISO 21175 specifies a technology-neutral message meta-model and associated exchange formats for collaborative distributed simulation on different manufacturing platforms within a Collaborative Modelling and Simulation Environment (CMSE).

It defines:

- an abstract message meta-model for the identification, addressing, timing and payload structure of messages exchanged during a simulation run;
- standardized message sets and their semantics for simulation lifecycle management, time management, model input and output exchange, monitoring, logging and error reporting;
- a normative JSON-based serialization binding for representing these messages in a machine-readable and interoperable form;
- general transport-binding principles for conveying such messages over message-oriented middleware or event-streaming infrastructures.

This document applies to collaborative simulation configurations in which a coordinator and a set of runners exchange messages in order to initialize, synchronize, execute, observe and terminate a joint simulation run. It is intended to support interoperability between heterogeneous simulation frameworks and platforms while preserving the semantics of logical time, message ordering and port-based interaction.

This document complements ISO 21175-1 by refining the message-based interaction mechanisms required to realize the Collaborative Modelling and Simulation Environment defined in that part.

The following are outside the scope of this document:

- the internal formalism, algorithms or implementation details of the simulation models executed by runners;
- the standardized representation, storage, discovery or reuse of models, which are addressed in ISO 21175-3;
- service interfaces, remote execution APIs, platform integration services and deployment procedures, which are addressed in ISO 21175-4;
- the selection of specific middleware products, network protocols, endpoint addresses or broker configurations;
- infrastructure-level concerns such as authentication, authorization, scaling, fault tolerance, persistence and resource provisioning, except where general interoperability requirements affect message exchange;
- application-specific payload semantics, transformation rules or domain ontologies beyond the message structures and typing principles defined in this document.

2 Normative references

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- ISO 21175-1, *Automation systems and integration — Collaboration Environment Requirements of Simulation on Different Manufacturing Platforms — Part 1: Reference model and process*.
- RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*.
- ISO/IEC 60559:2020, *Information technology — Microprocessor Systems — Floating-Point arithmetic*.

3 Terms and definitions

For the purposes of this document, the terms and definitions given in ISO 21175-1 and the following apply.

simulation message unit of information exchanged between participants in a collaborative simulation run within the CMSE

message meta-model abstract representation that defines the structure, attributes and relationships of simulation messages independently of any concrete serialization format or transport technology

simulation participant logical role or component involved in the exchange of simulation messages during a simulation run

message set coherent group of message types defined to support a specific aspect of the simulation protocol, such as lifecycle management, time management, model input/output exchange, monitoring or error handling

logical time representation of simulation time used to order events and coordinate the evolution of models during a simulation run

event time logical time at which a specific simulation event, output or transition is considered to occur

next internal event time logical time at which a model is scheduled to perform its next internal transition in the absence of external input

payload typed content carried by a simulation message

runner logical process that executes one subordinate simulation model and exchanges standardized messages with the coordinator

coordinator logical process that manages the progression of a collaborative simulation run by initializing runners, requesting outputs, issuing transition executions, collecting time information and terminating the run

4 Abbreviated terms

For the purposes of this document, the following abbreviated terms apply.

- **API** — Application Programming Interface;
- **CMSE** — Collaborative Modelling and Simulation Environment;
- **DEVS** — Discrete Event System Specification;

- **DCP** — Distributed Co-Simulation Protocol;
- **EF** — Experimental Frame;
- **FMI** — Functional Mock-up Interface;
- **IEEE** — Institute of Electrical and Electronics Engineers;
- **JSON** — JavaScript Object Notation;
- **OPM** — Object-Process Methodology;
- **RFC** — Request for Comments;
- **SSP** — System Structure and Parameterization;
- **UML** — Unified Modeling Language;
- **URI** — Uniform Resource Identifier;
- **UTF** — Unicode Transformation Format;
- **UUID** — Universally Unique Identifier.

5 Conformity with this part of ISO 21175

An implementation claiming conformity with this part of ISO 21175 shall conform to the message meta-model defined in Clause 7, to the semantics and constraints of the standard message sets defined in Clause 8, and to the JSON serialization rules defined in Clause 9, for all message types that it claims to support.

Conformity applies to logical components participating in collaborative distributed simulation, in particular coordinators, runners and, where implemented, monitoring or diagnostic components.

An implementation shall declare:

- the role or roles for which conformity is claimed;
- the message types supported by the implementation;
- the logical time encoding supported for a simulation run;
- any optional message types or optional fields supported by the implementation;
- any implementation-defined payload structures, monitoring categories or error code registries used in addition to the requirements of this document.

5.1 General conformance requirements

A conforming implementation shall satisfy the following requirements:

- it shall preserve the meaning of all mandatory message attributes defined in Clauses 7 and 8;
- it shall generate and interpret message identifiers, simulation run identifiers, sender identifiers and receiver identifiers in a manner consistent with 7.2;
- it shall represent logical time in a manner consistent with 7.3;

- it shall ensure that payloads use the primitive typing principles defined in 7.4;
- it shall encode messages in JSON in accordance with Clause 9 when claiming conformity to the normative serialization binding;
- it shall respect the ordering constraints defined in Clause 8 for each supported message set;
- it shall ignore unknown additional fields that do not alter the meaning of the mandatory fields defined by this document.

5.2 Conformance classes

This document defines the following conformance classes.

5.2.1 Coordinator conformance class

An implementation claiming conformity as a *coordinator* shall:

- support the generation and reception of lifecycle management messages as defined in 8.1;
- support the generation and reception of time management and scheduling messages as defined in 8.2;
- support the routing and interpretation of model input/output payloads as defined in 8.3;
- support the reception and handling of *ErrorReport* messages as defined in 8.5;
- send only messages whose event times and ordering are consistent with the latest time information received from runners.

5.2.2 Runner conformance class

An implementation claiming conformity as a *runner* shall:

- support the reception and processing of lifecycle management messages addressed to it, as defined in 8.1;
- support the reception and processing of time management and scheduling messages addressed to it, as defined in 8.2;
- generate *OutputReport*, *NextInternalTimeReport* and *TransitionComplete* messages in a manner consistent with the semantics of Clause 8;
- support the interpretation of input port values and the generation of output port values as defined in 8.3;
- report protocol or processing errors using *ErrorReport* as defined in 8.5.

5.2.3 Monitoring conformance class

An implementation claiming conformity as a *monitoring component* shall:

- support the generation or consumption of *MonitoringMessage* instances as defined in 8.4;
- not alter the execution semantics of the simulation run through the generation, modification or interpretation of monitoring messages;
- preserve the simulation run identifier and, when present, the logical time associated with monitored events or state observations.

5.3 Mandatory and optional features

The following features are mandatory for all conforming coordinator and runner implementations:

- support for the common message structure defined in Clause 9;
- support for the "SimulationInit", "SimulationTerminate", "NextInternalTimeReport", "RequestOutput", "OutputReport", "ExecuteTransition", "TransitionComplete" and "ErrorReport" message types;
- support for at least one agreed logical time encoding for a simulation run, as defined in 7.3.

The following features are optional:

- support for the "ModelTerminated" message type;
- support for the "MonitoringMessage" message type;
- support for additional implementation-specific fields that do not conflict with the requirements of this document.

An implementation shall not claim full conformity with this part of ISO 21175 if it omits any mandatory feature for the conformance class claimed.

5.4 Statement of conformity

A supplier or developer claiming conformity with this part of ISO 21175 shall provide a statement of conformity identifying:

- the conformance class or classes claimed;
- the edition of this part of ISO 21175 to which conformity is claimed;
- the list of supported message types;
- the supported logical time encoding or encodings;
- the optional features supported;
- any implementation-defined constraints relevant to interoperability.

5.5 Conformance testing

Conformance may be assessed by verifying that an implementation:

- produces messages whose structure and field values conform to Clauses 7, 8 and 9;
- correctly interprets valid incoming messages of the supported message types;
- rejects or reports invalid messages in accordance with 8.5;
- preserves the ordering and logical time constraints defined in this document for each simulation run and model identifier pair.

Detailed test procedures and test suites are outside the scope of this document.

6 General framework of collaborative message exchange

The general framework of collaborative message exchange defined in this clause provides a logical view of how simulation participants interact by means of standardized messages during a joint simulation run. It refines the general framework of the Collaborative Modelling and Simulation Environment (CMSE) introduced in ISO 21175-1 by focusing on the message-based interaction between simulation models, while remaining independent of any specific implementation technology or deployment of the underlying infrastructure.

In this framework, the simulation is organized around a *coordinator* process and a set of *model runners* (hereafter referred to as *runners*). The coordinator maintains the global view of the simulation run, including the list of participating models, their identifiers, their ports and the couplings between their ports. For the purpose of this part of ISO 21175, the simulation coordinator manages a set of runners. We will specify the communications requirements between the coordinator and the runners. This is sufficient because, the runners for subordinate models are assumed to be behaviorally equivalent. This allows construction of complex hierarchical models. For example, a single runner may control:

- an atomic DEVS model using a compliant DEVS framework
- a coupled DEVS model using a compliant DEVS framework
- a behaviorally equivalent simulation model using a compliant simulation framework not governed by DEVS
- another CMSE environment

For the purposes of this part of ISO 21175, interoperability is determined solely by the externally observable interaction behaviour of a runner at the message interface. This behaviour includes event-based input and output exchange, logical-time information, and the ordering behaviour required by the coordinator–runner protocol defined in this document. The internal formalism, algorithms and execution mechanisms of the subordinate model remain outside the scope of this document.

Any subordinate system executed by a runner shall be considered interoperable within the meaning of this document if, either directly or by means of an adapter or wrapper, it conforms to the externally observable interaction behaviour required by the coordinator–runner protocol. Such a subordinate system may be an atomic DEVS model, a coupled DEVS model, a model expressed in another formalism (including discrete-time, continuous-time or hybrid formalisms), or another simulation environment.

From a system-theoretic perspective, a subordinate system satisfying these interface conditions may be regarded as a DEVS-like system, that is, a system with DEVS-compatible input and output interfaces. By the universality and uniqueness results established for DEVS, every such system admits a canonical realization as a DEVS input/output system [2, 6, 7]. Accordingly, for the purposes of collaborative distributed execution, this document standardizes a single protocol-level interaction structure expressed in DEVS-compatible terms.

Consequently, a non-DEVS, non-modular or otherwise heterogeneous model that is integrated through an adapter or wrapper is not treated as an alternative protocol-level model representation. Instead, it is treated through the same canonical DEVS-compatible interaction structure. Its native internal representation, its internal organization, and the internal design of the adapter or wrapper are not specified by this document. Only the externally visible message behaviour of the resulting runner is normative.

Each runner is responsible for executing its subordinate model in accordance with the semantics of the formalism it encapsulates, provided that the messages it sends and receives conform to the coordinator–runner protocol defined in this part of ISO 21175. The overall simulation

behaviour is obtained by coordinating such runners through standardized message exchange [3, 4, 5].

6.1 Roles in message-based collaborative simulation

Within this general framework, the following roles are distinguished:

- **Coordinator:** a logical process that controls and synchronizes the joint simulation. It does not execute model code itself, but orchestrates the execution of runners by sending them control messages and by routing their inputs and outputs according to the couplings defined for the joint simulation configuration. The coordinator holds a unique list of model identifiers, where each identifier refers to exactly one subordinate model executed by exactly one runner.
- **Runner:** a logical process that encapsulates a subordinate simulation model. A runner has access to the model's state and functions (e.g. state transition functions, output function and time advance function), but it does not have direct knowledge of other models, of any hierarchical structure, or of the overall simulation configuration. Its only dependency on the rest of the system is the ability to send and receive messages via the message exchange mechanism.

These roles are logical roles. A concrete implementation may map several roles to a single executable process, or deploy them on different machines or platforms, without changing the conceptual framework defined in this clause.

6.2 Logical communication topology

Note: One single communication protocol The coordinator and the runners shall communicate via a message-oriented middleware or event streaming mechanism that provides asynchronous message delivery on a single shared logical channel. The middleware shall preserve message order on that logical channel for a given simulation run.

In this communication pattern:

- all participants, including the coordinator and all runners, publish messages to the same shared logical channel;
- all participants consume messages from that same shared logical channel;
- the coordinator shall process only those messages whose `receiverId` matches its assigned identifier, and shall ignore all other messages;
- each runner shall process only those messages whose `receiverId` matches its assigned identifier, and shall ignore all other messages.

This communication pattern ensures that:

- a single ordered message stream defines the observable execution history of the collaborative simulation;
- all participants operate over a consistent view of message ordering;
- the standard remains independent of any specific transport product or infrastructure technology.

The concrete realization of the shared logical channel is an implementation concern. Depending on the chosen infrastructure, it may be mapped to a topic, queue, stream or equivalent communication construct, provided that the ordering and addressing requirements defined in this document are preserved.

6.3 Relation to business and system description

From the perspective of ISO 21175-1, the coordinator embodies the coupled simulation model defined at the software collaboration level. It holds the description of the joint simulation structure, including:

- the list of subordinate models participating in the joint simulation and their identifiers;
- the ports exposed by each subordinate model and their types and roles;
- the couplings that route outputs from source ports of subordinate models to inputs of other subordinate models;
- the association between each subordinate model and the runner that executes it.

In the message-based framework, each subordinate model in the hierarchy is represented by one runner, and the couplings between higher-level components are resolved into couplings between ports of these subordinate models. These relationships are reflected by the configuration of the coordinator and by the routing of messages between runners through the message infrastructure.

Runners, in contrast, only require knowledge of the subordinate model they execute and its local interface (ports and internal state). They do not need to know which other models exist, how they are connected, or whether the model was originally defined as part of a hierarchical coupled model. This separation of concerns enables the independent development, deployment and reuse of models across different joint simulation projects.

6.4 Simulation lifecycle and message flow

The message exchange during a joint simulation run follows a well-defined lifecycle. While the exact message types are specified in Clause 8, the main phases and patterns are as follows:

Initialization phase. In the initialization phase, the coordinator ensures that all runners are ready to participate in the simulation and that they share a common initial logical time:

- the coordinator identifies the set of subordinate models and corresponding runners that belong to the simulation configuration;
- for each runner, the coordinator sends an initialization message on the runner's input channel, indicating, at a minimum, the initial logical time and the identifier of the model to execute;
- each runner initializes the associated model (state and internal time variables) and responds with a message indicating the time of its next internal event according to the model's time advance function.

Once the coordinator has received a response from each runner, it determines the earliest next event time among all runners and sets the global simulation time accordingly.

Execution phase. During the execution phase, the coordinator repeatedly advances the simulation time and directs runners to execute internal, external or confluent transitions and to produce outputs. The general pattern is:

- the coordinator identifies the current global simulation time as the minimum of the next event times reported by the runners;

- for each runner that is imminent at this time, the coordinator will first request outputs by sending an output request message; the runner executes the model's output function and returns any generated outputs in a corresponding output message;
- the coordinator routes the outputs to the inputs of affected models according to the couplings defined in the configuration, preparing input data for those models;
- the coordinator then sends transition execution messages to the relevant runners:
 - a transition message without inputs corresponds to an internal transition at the runner's scheduled next event time;
 - a transition message with inputs at a time earlier than the runner's next internal event corresponds to an external transition;
 - a transition message with inputs at the same time as the runner's next internal event corresponds to a confluent transition.
- each runner executes the requested transition on its model, updates its internal state and next event time, and responds with a completion message that supplies the new next event time to the coordinator.

This interaction pattern yields a parallel discrete-event simulation: all subordinate models conceptually evolve in parallel under the supervision of the coordinator, which advances the global logical time from one event to the next based solely on information obtained from the runners through messages.

Termination phase. The termination phase concludes a simulation run when a stopping condition is met (for example, a maximum logical time, an absence of further events, or a condition detected by external supervision or monitoring logic):

- the coordinator determines that the stopping condition is satisfied;
- it sends a termination message to all runners, instructing them to finalize the current run and release any resources held for that run;
- runners acknowledge termination and may emit final result or summary messages before stopping or returning to an idle state.

TODO: merge monitoring with execution phase and collection with termination phase, i am not sure monitoring is completely independant

Monitoring and result collection. Optionally, runners and the coordinator can emit additional monitoring and logging messages during initialization and execution, for example:

- periodic state snapshots for selected models;
- event traces for debugging and validation;
- aggregated measures of performance or effectiveness computed during the run.

These messages are typically published on one or more logging channels, which may be shared among runners or organized according to implementation needs. State snapshots are only needed at discrete change points (initialization and transitions), since the state of a discrete-event model does not change between these events. The creation of snapshots and the selection of variables to be included are responsibilities of the runner implementation and are not mandated by this part of ISO 21175.

6.5 Independence from infrastructure deployment

The framework defined in this clause describes the message-based behaviour of a joint simulation independently of any particular infrastructure or deployment choice. In particular:

- the coordinator and the runners are treated as logical processes, which may be co-located on a single machine or distributed across multiple platforms;
- the message-oriented middleware or event streaming mechanism is treated as an abstract communication service; this part of ISO 21175 does not require the use of any specific product or protocol. However, for any given joint simulation run, all participating components (including the coordinator and all runners) *shall* use the same message-oriented middleware or event streaming mechanism, or a set of interoperable instances thereof, so that messages conforming to this part can be exchanged consistently;
- details related to service endpoints, authentication, load balancing, scaling or fault tolerance are out of scope of this part and may be addressed in future parts dealing with service and API specifications.

Subsequent clauses of this part specify the message meta-model and standard message sets that realize this framework, while other parts of ISO 21175 address the model representations and service interfaces that build upon this

TODO: Clarify in an informative note how implementers may introduce additional experimental or analysis components (for example, logic embedded in the coordinator or external tools consuming logging channels) without introducing new roles or message types beyond those defined in this part.

Note: Specification concerning IP and topic for kafka for example will be explained in the Part 4 of the ISO, we try to be as much as generic as possible here

7 Message meta-model

This clause specifies an abstract meta-model for simulation messages, independent of any concrete serialization format or transport mechanism. The meta-model identifies the core concepts (messages, models, ports, payloads and related identifiers) and their relationships, so that message sets defined in Clause 8 can be expressed in a consistent and implementation-neutral way.

The message meta-model is intended for subordinate models participating in a joint simulation configuration as described in Clause 6. Higher-level or hierarchical model structures defined at the modelling level are mapped, prior to execution, to the subordinate models and ports represented in this meta-model.

Textual definitions in this clause are normative. Diagrams based on this meta-model (for example, OPM, X Language or UML class diagrams) may be provided in an informative annex to support understanding and implementation.

7.1 Core concepts and structural elements

This subclause defines the main concepts of the message meta-model and how they relate to each other. The intent is to provide a small set of well-defined elements that can be mapped to different technologies without changing the semantics of the simulation.

The following concepts are defined:

- **Simulation run:** a single execution of a joint simulation configuration, identified by a *simulation run identifier*. All messages belonging to the same execution share this identifier.

- **Model:** a subordinate simulation entity with a well-defined interface and behaviour. In this part of ISO 21175, a model:
 - has a unique *model identifier* within a given simulation run;
 - exposes a set of *ports* through which it exchanges messages;
 - is executed by exactly one runner in the framework defined in Clause 6.
- **Port:** a logical interface of a model used to send or receive messages. A port:
 - has a *port name* that is unique within the scope of its model;
 - has a *direction* (input or output);
 - is associated with one or more *payload types* indicating the primitive types (see 7.4) that may legally appear in messages sent or received via that port.
- **Message:** the base concept representing a unit of information exchanged between participants in a simulation run. A message:
 - is identified by a *message identifier* that is unique within the scope of the simulation run;
 - includes the simulation run identifier;
 - has a *message type* chosen from a controlled vocabulary defined by the standard message sets in Clause 8 (for example, initialization message, output message, transition completion message);
 - carries one or more *time attributes* (see 7.3), such as the current logical time of the event and, where applicable, the next internal event time as computed by the model (often denoted t and t_{next});
 - identifies the *sender model* (or coordinator) by its model identifier or by a reserved identifier for the coordinator;
 - identifies a *receiver model* or runner by its model identifier;
 - may include *correlation information* used to relate responses to requests or to group messages belonging to the same logical transaction (for example, a correlation identifier);
 - contains a *payload* whose structure and primitive types are specified according to 7.4.
- **Payload:** the typed content carried by a message. A payload:
 - is composed of values built from a restricted set of primitive data types (for example, integer, floating-point number, string, Boolean) and, optionally, structured “object” values encoded using these primitive types;
 - is interpreted according to the message type and, when relevant, according to the port type on which the message is conceptually emitted or received.
- **Context:** information describing the joint simulation project and the configuration under which a simulation run is executed (for example, identifiers of collaborating organizations, scenario description, configuration version). Context information is associated with the simulation run and may be referenced in messages when needed, but it is not required for every message.

Conceptually, each message belongs to exactly one simulation run, is produced by exactly one sender (coordinator or model) and may be associated with one or more ports depending on the semantics of the message set (for example, an output message is associated with a single output port of a model, while a control message from the coordinator may be associated with no port).

7.2 Identification and addressing

This subclause specifies how models, ports, simulation runs and message streams are identified so that they can interoperate across platforms in a consistent way.

Each simulation run *shall* be identified by a *simulation run identifier* that is unique at least within the scope of the message-oriented middleware or event streaming mechanism used for that run. All messages belonging to the same execution *shall* carry this identifier.

Each model participating in a given simulation run *shall* be identified by a *model identifier* that is unique within that simulation run. The coordinator will also have its own unique identifier. To avoid confusion when the same model is reused by different organizations or in different runs, it is recommended that model identifiers be based on universally unique identifiers (UUIDs), URIs or other schemes with a low probability of collision, instead of relying solely on human-readable names. Human-readable model names may be provided in addition to model identifiers but are not sufficient for addressing.

Each port of a model *shall* have a *port name* that is unique within the scope of that model. Port names are primarily used to identify which part of the model interface a given message relates to (for example, which input or output port is being used) and are interpreted in combination with the model identifier. The combination of (*simulation run identifier*, *model identifier*, *port name*) is therefore unique for the duration of the simulation run.

In the framework defined in Clause 6, addressing between the coordinator and runners is realized via logical channels (for example, topics or queues). For the purposes of this meta-model:

- the mapping between model identifiers and specific channels is an infrastructure concern and is not fixed by this part of ISO 21175;
- messages *shall* contain sufficient identification information (at least the simulation run identifier and the sender model or coordinator identifier, and receiver identifier) so that they can be interpreted independently of the particular channel over which they are transported.

The specific encoding of these identifiers (for example, UUID format, numeric identifiers, URIs) is determined by the serialization and transport bindings in Clause 9, provided that uniqueness and traceability requirements stated in this subclause are satisfied.

7.3 Time representation

This subclause defines how logical time is represented in the message meta-model. Logical time is used to order events and to determine the evolution of models in a discrete-event simulation.

The meta-model distinguishes between:

- *event time*: the logical time at which a given event (such as an internal or external transition) occurs;
- *next internal event time*: the logical time at which a model intends to perform its next internal transition in the absence of external inputs, as computed by its time advance function;
- optionally, *wall-clock time*: the physical time at which a message is sent or received, used only for monitoring, logging or real-time considerations.

For the purposes of interoperability, logical time values used in messages *shall* be encoded using primitive numeric types as defined in 7.4. Implementations *shall* support the following encodings for logical time:

- a 64-bit floating-point type following the IEEE 754 double-precision format (`float64`).

Logical time infinity (that is, the absence of further events) *shall* be represented, for a given encoding, by a distinguished value. When 64-bit floating-point encoding is used, this value *shall* correspond to the maximum representable value.

Hybrid time representations (for example, tuples combining a discrete counter and a continuous time value) may be used by representing each component as a primitive type and grouping them into an object payload, as described in 7.4. The ordering of such hybrid times *shall* be well-defined and consistently interpreted by all components participating in a simulation run.

The semantics of the simulation (for example, Parallel DEVS semantics) are independent of the exact numeric encoding used for logical time, as long as the ordering of events is preserved.

7.4 State and payload description

This subclause specifies how the values carried in message payloads and, where applicable, snapshots of model state are described in the meta-model.

To promote interoperability across different programming languages and platforms, payload values *shall* be expressed using a restricted set of *primitive data types* that can be represented and cast in a wide range of environments. At a minimum, the following primitive types are considered:

- **boolean**: logical truth values;
- **int64**: signed integers of maximum 64 bits respectively;
- **float64**: floating-point values following IEEE 754 double precision formats;
- **string**: sequences of characters encoded in a specified character set (for example, UTF-8);
- **object**: A valid Json object which only contains primitives values, JSON Object or JSON Array.
- **array**: A valid Json array which only contains primitives values, JSON Object or JSON Array.

The exact subset of these primitive types that must be supported and their mapping to concrete encodings are defined in Clause 9. Additional primitive types may be introduced in future revisions, provided that they can be mapped unambiguously to widely supported language-level types.

Structured payloads are represented as *objects* built from primitive types. In this case:

- the payload is treated as an *object* containing named fields;
- each field has an associated primitive type (for example, an **int64** for a counter, a **float64** for a measurement, a **string** for a label);
- the object is serialized using a structured format based on primitive values, such as JSON, as specified in Clause 9.

The message meta-model does not impose a global schema language or ontology for payloads. Instead, it requires that:

- for each message type defined in Clause 8, the expected payload structure (fields and primitive types) is specified in the standard or in a referenced specification;
- for each port, the set of primitive types or object structures that may appear in messages on that port is documented, so that implementers can perform type checking and casting when sending or receiving messages.

State snapshots, when used for monitoring, debugging or analysis, *shall* reuse the same primitive and object mechanisms. A snapshot therefore consists of an object whose fields represent selected state variables of a model at a given logical time, expressed using primitive types and, if needed, nested objects. As the models considered in this part are discrete-event models, state snapshots are only meaningful at discrete change points (for example, initialization and transitions) and are not required between such events.

The selection of which state variables to include in snapshots, and how often to generate them, is the responsibility of the runner implementation. This part of ISO 21175 does not mandate any particular set of state variables or snapshot frequency.

7.5 Relationship to business and system description

This subclause links the message meta-model defined above to the business and system description concepts introduced in ISO 21175-1.

At the business level, ISO 21175-1 describes processes, resources and collaborating parties involved in a joint simulation project. At the system and software collaboration levels, it identifies the simulation models and components that represent these processes and resources, as well as their interactions.

In the context of this part:

- each subordinate simulation model represented in the message meta-model corresponds to the implementation of a business or system element identified in the business & system description (for example, a machine, a buffer, a controller or an information system);
- each port of a model corresponds to a specific interaction point or interface of that element (for example, a material flow, a control signal, an information message);
- each message type defined in Clause 8 corresponds to a specific kind of interaction in the simulation (for example, a request to execute a transition, a notification of outputs, a report of the next internal event time).

The mapping from business processes and system architectures to models and ports is defined as part of the CMSE configuration in ISO 21175-1. The message meta-model ensures that, once this mapping is established, the interactions between models can be expressed as messages that:

- are unambiguously associated with the models and ports involved;
- carry payloads whose primitive types and structures are compatible with the intended domain semantics;
- can be traced back to business and system requirements when needed (for example, for validation, verification and audit).

This traceability supports the reuse of models across different joint simulation projects, the analysis of simulation results in relation to business objectives, and the verification that message exchanges conform to the scenarios and requirements specified at the business and system levels.

8 Standard message sets for collaborative simulation

This clause defines standard message sets that realize the general framework for collaborative message exchange described in Clause 6, using the message meta-model specified in Clause 7. Each message set groups together message types that serve a coherent simulation purpose (for example, simulation lifecycle management, time management, or model input/output).

Each concrete message defined in this clause belongs to one *message type*. To support generic processing and filtering in implementations, each message *shall* carry a *messageType* attribute whose value is a string chosen from a controlled vocabulary defined by this standard.

Table 1 summarizes the standard *messageType* identifiers introduced in this clause.

messageType value	Abstract message type
"SimulationInit"	Simulation initialization message
"SimulationTerminate"	Simulation termination message
"ModelTerminated"	(Optional) Acknowledge completion of termination steps
"NextInternalTimeReport"	Report of next internal event time
"RequestOutput"	Request to evaluate model output function
"OutputReport"	Report of model outputs at a given event time
"ExecuteTransition"	Request to execute a model transition
"TransitionComplete"	Acknowledgement of completed transition
"MonitoringMessage"	Monitoring, logging or result reporting message
"ErrorReport"	Error and diagnostic message

Table 1: Standard messageType identifiers.

Unless otherwise stated, all messages defined in this clause:

- *shall* conform to the meta-model specified in Clause 7;
- *shall* carry a simulation run identifier and a message identifier as defined in 7.2;
- *shall* carry a *messageType* attribute whose value is exactly one of the strings listed in Table 1;
- *shall* identify their sender (coordinator or model) and, their associated receiver model identifier, and, when relevant port name(s);
- *shall* carry at least one logical time attribute as defined in 7.3 when they relate to a simulation event.

The *messageType* attribute is an abstract attribute of the message meta-model. Concrete encodings (for example, JSON) *shall* provide a field or mechanism that preserves its value without loss of information. Clause 9 defines one such encoding.

The ordering rules given below are expressed for each pair (*simulation run identifier*, *model identifier*). Implementations may run several simulation runs in parallel, or reuse a runner in successive runs, provided that the ordering constraints are respected independently for each simulation run.

8.1 Simulation lifecycle management messages

The simulation lifecycle management message set covers the initialization and termination of a simulation run for a given runner. Creation and destruction of the simulation run itself (for example, allocation of computing resources) are part of the CMSE configuration and the service interfaces defined in other parts of ISO 21175 and are out of scope of this message set.

8.1.1 SimulationInit

The *SimulationInit* message is sent by the coordinator to a runner to initialize a subordinate model for participation in a given simulation run.

For a *SimulationInit* message:

- the *messageType* attribute *shall* have the value "SimulationInit";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the identifier of the sending coordinator;
 - the identifier of the receiving model to be executed by the runner;
 - the initial logical event time of the simulation run (for example, t_0).

Model instantiation and parameterization are handled by the CMSE configuration and by the service interfaces defined in other parts of ISO 21175. The *SimulationInit* message is limited to initializing the execution of a model that has already been instantiated and parameterized.

Upon receipt of *SimulationInit*:

- the runner *shall* initialize the associated model state and any internal time variables for the given simulation run;
- the runner *shall* compute the next internal event time according to the model's time advance function, using the initial event time;
- the runner *shall* respond with a *NextInternalTimeReport* message (see 8.2) reporting the next internal event time.

8.1.2 SimulationTerminate

The *SimulationTerminate* message is sent by the coordinator to a runner to indicate the end of a simulation run and to release resources associated with that run.

For a *SimulationTerminate* message:

- the *messageType* attribute *shall* have the value "SimulationTerminate";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the identifier of the sending coordinator;
 - the model identifier of the receiving runner;
 - a logical event time at which the termination is considered effective;
- the message *may* contain a termination reason, expressed as a primitive value or code as defined in 7.4.

Upon receipt of *SimulationTerminate*:

- the runner *shall* finalize any processing related to the given simulation run, release any associated state and resources, and cease to send further time management or model input/output messages for that run;
- As agreed upon by the CMSE configuration, the runner may send a "ModelTerminated" message to indicate completion of termination steps.
- the runner *may* send final monitoring or result reporting messages as defined in 8.4;
- the runner *may* send an *ErrorReport* message as defined in 8.5 if an error occurs during termination.

8.1.3 ModelTerminated

The optional *ModelTerminated* message is sent by a subordinate model in response to a *SimulationTerminate* request. This is useful in cases where the CMSE participants may have shared infrastructure, such as a common log storage system, and the coordinator needs to wait for all participants to terminate before stopping the infrastructure.

For a *ModelTerminated* message:

- the *messageType* attribute shall have the value "ModelTerminated";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the identifier of the sending runner;
 - the identifier of the receiving coordinator;

Upon receipt of *ModelTerminated*:

- the runner shall finalize any processing related to the given simulation run, release any associated state and resources, and cease to send further time management or model input/output messages for that run. It will stop any shared infrastructure ;

8.1.4 Ordering constraints for lifecycle messages

For each pair of identifiers (*simulation run identifier*, *model identifier*):

- the coordinator *shall not* send any time management or model input/output messages to the runner for that simulation run before a *SimulationInit* message has been sent for that pair;
- the runner *shall not* send any time management or model input/output messages for that simulation run before it has received a *SimulationInit* message for that pair;
- exactly one *SimulationInit* message *shall* be sent by the coordinator to a given runner for a given (*simulation run identifier*, *model identifier*) pair;
- after a *SimulationTerminate* message has been sent by the coordinator for a given (*simulation run identifier*, *model identifier*) pair, no further time management or model input/output messages *shall* be sent by the coordinator or by the runner for that pair;
- if the CMSE configuration requires *ModelTerminated* messages before shared infrastructure is shut down, after a runner receives a *SimulationTerminate*, after it has completed final processing and no longer needs shared infrastructure, it shall send a *ModelTerminated* message to the coordinator.
- if a runner receives a time management or model input/output message for a simulation run for which it has not yet received a *SimulationInit* message, or for which it has already received a *SimulationTerminate* message, it *shall* treat this as an error and *shall* report it using an *ErrorReport* message as defined in 8.5.

8.2 Time management and scheduling messages

The time management and scheduling message set allows the coordinator to synchronize the evolution of all subordinate models in a parallel discrete-event simulation as described in Clause 6. It provides the minimal set of messages required to:

- initialize logical time at each runner;
- query and report the next internal event time of each model;
- request execution of internal, external and confluent transitions at specific logical times;
- acknowledge completion of transitions and report updated next internal event times.

8.2.1 NextInternalTimeReport

The *NextInternalTimeReport* message is sent by a runner to the coordinator to report the next internal event time of the model it executes for a given simulation run.

For a *NextInternalTimeReport* message:

- the *messageType* attribute *shall* have the value "NextInternalTimeReport";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the sending model identifier;
 - the identifier of the receiving coordinator;
 - the current event time at which the report is issued;
 - the next internal event time as computed by the model's time advance function (or the distinguished infinity value as defined in 7.3 if no further internal events are scheduled).

A *NextInternalTimeReport* message may be sent:

- in response to a *SimulationInit* message, to report the first next internal event time after initialization;
- after completion of a transition, as described for *TransitionComplete* in 8.2.5, if the message set profile chosen for the simulation run requires a separate next-time report.

8.2.2 RequestOutput

The *RequestOutput* message is sent by the coordinator to a runner to request that the runner execute the model's output function at a specified event time and report any resulting outputs.

For a *RequestOutput* message:

- the *messageType* attribute *shall* have the value "RequestOutput";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the identifier of the sending coordinator;
 - the model identifier of the receiving runner;
 - the event time at which the output function is to be evaluated.

The event time contained in a *RequestOutput* message *shall* be equal to the latest next internal event time reported by the runner for that simulation run.

8.2.3 OutputReport

The *OutputReport* message is sent by a runner to the coordinator in response to a *RequestOutput* message. It carries the outputs produced by the model's output function at a given event time.

For an *OutputReport* message:

- the *messageType* attribute *shall* have the value "OutputReport";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the sending model identifier;
 - the identifier of the receiving coordinator;
 - the event time at which the output function has been evaluated;
 - zero or more *port values*, each consisting of:
 - * the name of an output port of the model;
 - * a payload object containing one or more named primitive values, as defined in 7.4.

An *OutputReport* message *may* also contain the next internal event time as computed by the model after the output function evaluation, if the message set profile chosen for the simulation run allows reporting of the next internal time together with outputs.

All port values contained in a single *OutputReport* message *shall* be associated with the same event time.

8.2.4 ExecuteTransition

The *ExecuteTransition* message is sent by the coordinator to a runner to request that the runner execute a state transition of its model at a specified event time.

For an *ExecuteTransition* message:

- the *messageType* attribute *shall* have the value "ExecuteTransition";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the identifier of the sending coordinator;
 - the model identifier of the receiving runner;
 - the event time at which the transition shall occur;
 - zero or more *input port values*, each consisting of:
 - * the name of an input port of the model;
 - * a payload object containing one or more named primitive values, as defined in 7.4.

The semantics of *ExecuteTransition* messages are as follows:

- if no input port values are present and the event time is equal to the latest next internal event time reported by the runner, the message corresponds to an *internal transition*;
- if one or more input port values are present and the event time is strictly less than the latest next internal event time reported by the runner, the message corresponds to an *external transition*;
- if one or more input port values are present and the event time is equal to the latest next internal event time reported by the runner, the message corresponds to a *confluent transition*.

All input port values contained in a single *ExecuteTransition* message *shall* be associated with the same event time.

8.2.5 TransitionComplete

The *TransitionComplete* message is sent by a runner to the coordinator to acknowledge completion of a transition requested by an *ExecuteTransition* message and to report the updated next internal event time.

For a *TransitionComplete* message:

- the *messageType* attribute *shall* have the value "TransitionComplete";
- the message *shall* contain at least:
 - the simulation run identifier;
 - the sending model identifier;
 - the identifier of the receiving coordinator;
 - the event time at which the transition has occurred;
 - the new next internal event time, or the distinguished infinity value as defined in 7.3 if no further internal events are scheduled.

A *TransitionComplete* message *shall* be sent exactly once in response to each *ExecuteTransition* message that has been successfully processed by the runner, except in the presence of errors reported via *ErrorReport* messages as defined in 8.5.

8.2.6 Ordering constraints for time management messages

For each (*simulation run identifier*, *model identifier*) pair:

- after receipt of a *SimulationInit* message, the runner *shall* send one and only one *NextInternalTimeReport* message before sending any *OutputReport* or *TransitionComplete* messages for that simulation run;
- each *RequestOutput* message sent by the coordinator *shall* refer to an event time that is equal to the latest next internal event time reported by the runner;
- a runner *shall not* send an *OutputReport* message unless it has previously received a corresponding *RequestOutput* message for the same simulation run and model identifier and for the same event time;
- for each *RequestOutput* message sent by the coordinator, the runner *shall* send exactly one *OutputReport* message, except in the presence of errors reported via *ErrorReport* messages;
- a runner *shall not* send a *TransitionComplete* message unless it has previously received a corresponding *ExecuteTransition* message for the same simulation run and model identifier and for the same event time;
- for each *ExecuteTransition* message sent by the coordinator, the runner *shall* send exactly one *TransitionComplete* message, except in the presence of errors reported via *ErrorReport* messages;
- the sequence of (*event time*, *next internal event time*) pairs reported by a runner via *NextInternalTimeReport*, *OutputReport* (when it carries a next internal event time) and *TransitionComplete* *shall* preserve the ordering semantics of logical time defined in 7.3: the event times reported *shall* form a non-decreasing sequence, and the next internal event time reported after a transition *shall* be greater than or equal to the event time of that transition.

These constraints do not prescribe a particular scheduling algorithm for the coordinator. Implementations may choose different strategies for selecting the next event time or for batching requests, provided that the ordering rules above and the semantics of the underlying formalism are preserved.

8.3 Model input and output messages

Model input and output messages convey domain-level information between models through their ports. They are used in conjunction with time management messages to achieve the behaviour described in Clause 6.

In the message meta-model defined in Clause 7, domain-level information is carried as payload values associated with ports. This subclause specifies additional constraints on the port-level payloads used in *OutputReport* and *ExecuteTransition* messages.

8.3.1 Port values in *OutputReport* messages

Each port value included in an *OutputReport* message *shall* conform to the following:

- the port name *shall* identify an output port of the model as declared in the model representation defined in ISO 21175-3;
- the associated payload object *shall* be composed of one or more named primitive values, using only the primitive data types defined in 7.4;
- all port values in a single *OutputReport* message *shall* share the same event time, which is the event time of the message.

The coordinator *shall* route each port value from an *OutputReport* message to the input ports of other models according to the couplings defined in the joint simulation configuration. This routing may involve transformation of payloads, but any such transformations are out of scope of this part of ISO 21175.

8.3.2 Port values in *ExecuteTransition* messages

Each input port value included in an *ExecuteTransition* message *shall* conform to the following:

- the port name *shall* identify an input port of the model as declared in the model representation defined in ISO 21175-3;
- the associated payload object *shall* be composed of one or more named primitive values, using only the primitive data types defined in 7.4;
- all input port values in a single *ExecuteTransition* message *shall* share the same event time, which is the event time of the message.

When several output port values from different models are routed to the same input port of a model at the same event time, the coordinator *shall* define a deterministic policy for combining these values into the input port values to be included in the *ExecuteTransition* message (for example, aggregation into a list or selection according to a specified rule). The choice of combination policy is out of scope of this part of ISO 21175 but *shall* be documented for each implementation.

8.3.3 Ordering constraints for model input and output

Model input and output messages appear as payloads within *OutputReport* and *ExecuteTransition* messages. The ordering constraints defined in 8.2 therefore apply to them implicitly.

Additionally, for each (*simulation run identifier*, *model identifier*) pair:

- a runner *shall* interpret the port values contained in an *ExecuteTransition* message as all inputs available to the model at the event time of that message; inputs arriving at different event times *shall* be conveyed in separate *ExecuteTransition* messages;
- a runner *shall* not rely on any particular ordering of input port values within a single *ExecuteTransition* message; if the ordering of multiple values is semantically significant, it *shall* be represented explicitly in the payload object (for example, as an indexed collection).

8.4 Monitoring, logging and result reporting messages

Monitoring, logging and result reporting messages are used to observe the behaviour of a simulation run without affecting its execution semantics. They may be consumed by external tools (for example, dashboards or analysis components) and are typically routed via separate channels from the core coordination messages, as described in Clause 6.

This part of ISO 21175 defines a generic *MonitoringMessage* type that can be specialized or extended by implementers. The use of these messages is optional; conforming implementations *may* choose not to support them.

8.4.1 MonitoringMessage

For a *MonitoringMessage*:

- the *messageType* attribute *shall* have the value "MonitoringMessage";
- the message *shall* contain at least:
 - the simulation run identifier;
 - a source role identifier indicating whether the sender acts as a coordinator, runner or another participant;
 - a source identifier, which, when the source role is runner, *shall* be the model identifier of the model executed by that runner;
 - a receiver identifier which identifies the receiver (coordinator or subordinate model) of the message by its identifier.
- the message *may* contain:
 - a logical time if the observation is associated with a specific simulation event;
 - a category, expressed as a string or an enumeration, indicating the nature of the observation (for example, *stateSnapshot*, *metric*, *trace*, *debug*);
 - a list of one or more named primitive values, each consisting of:
 - * a name;
 - * a primitive type as defined in 7.4;
 - * a primitive value of the indicated type.

The contents and interpretation of the named primitive values depend on the category. Examples include:

- for category *stateSnapshot*, named values may represent selected state variables of the model at the given logical time;
- for category *metric*, named values may represent aggregated measures of performance or effectiveness computed over part or all of the simulation run;
- for category *trace*, named values may represent individual events of interest (for example, arrivals, departures, or threshold crossings).

This part of ISO 21175 does not mandate a specific set of categories or variables to be reported. The selection of categories and values is the responsibility of the implementation and may be agreed among collaborating parties.

8.4.2 Result reporting

Result reporting for an experimental frame or for a complete simulation run may be implemented using one or more *MonitoringMessage* instances, for example with a dedicated category such as *runSummary*. Alternatively, implementations may provide separate messages that reference result sets stored in external repositories; such messages *shall* reuse the same primitive and object mechanisms defined in 7.4.

The choice of result storage mechanisms (for example, databases or files) and their access interfaces is out of scope of this part of ISO 21175.

8.4.3 Ordering constraints for monitoring and logging

Monitoring, logging and result reporting messages do not participate in the simulation state evolution. For each (*simulation run identifier*, *model identifier*) pair:

- a runner or coordinator *may* emit *MonitoringMessage* instances at any time after *SimulationInit* and before *SimulationTerminate* for that pair;
- the relative ordering of *MonitoringMessage* instances with respect to time management and model input/output messages is not constrained by this part of ISO 21175; however, when a logical time is included in a *MonitoringMessage*, it *shall* reflect the logical time of the event or state being observed.

8.5 Error handling and diagnostic messages

Error handling and diagnostic messages are used to report exceptional conditions that occur during a simulation run. They enable participants to take appropriate decisions (for example, to attempt recovery, to terminate the run or to ignore non-critical issues).

This part of ISO 21175 defines a generic *ErrorReport* message type. The use of this message type is mandatory for conforming implementations that need to report errors related to the simulation protocol.

8.5.1 ErrorReport

For an *ErrorReport* message:

- the *messageType* attribute *shall* have the value "ErrorReport";
- the message *shall* contain at least:
 - the simulation run identifier, when the error is associated with a specific simulation run; for errors not associated with any simulation run, this field *may* be omitted;

- an origin role identifier indicating whether the sender acts as a coordinator, runner or another participant;
 - an origin identifier, which, when the origin role is runner, *shall* be the model identifier of the model executed by that runner;
 - a receiver identifier which identifies the receiver (coordinator or subordinate model) of the message by its identifier.
 - a severity level, chosen from the following set:
 - * *info*: informational condition that does not indicate a malfunction;
 - * *warning*: condition that may indicate a potential problem but does not immediately compromise the correctness of the simulation run;
 - * *error*: condition that may compromise the correctness of the current simulation run but does not prevent continued operation of the component;
 - * *fatal*: condition that prevents the component from continuing the current simulation run.
 - an error code, expressed as a string or integer, identifying the category of error according to an implementation-defined or community-defined registry;
 - a human-readable error message describing the condition.
- the message *may* contain additional details encoded as one or more named primitive values, as defined in 7.4.

8.5.2 Use of *ErrorReport* in the protocol

When an error is detected in relation to a specific protocol message (for example, a malformed message, an unexpected message type or a violation of ordering constraints), the receiving component *shall* report it using an *ErrorReport* message, unless the error prevents it from sending any further messages.

The recommended reactions to different severity levels are as follows:

- for severity *info*, no protocol-level action is required;
- for severity *warning*, the recipient *may* log the condition and continue the simulation run;
- for severity *error*, the recipient *should* consider terminating the current simulation run for the affected models and *may* send a *SimulationTerminate* message with an appropriate termination reason;
- for severity *fatal*, the originator indicates that it cannot continue the current simulation run; if the coordinator receives a *fatal ErrorReport* for any participant in a given simulation run, it *shall* terminate that simulation run and *shall* send *SimulationTerminate* messages to all runners participating in that run.

Errors that occur during processing of *SimulationInit*, *SimulationTerminate*, time management or model input/output messages *shall* be reported using *ErrorReport*. This part of ISO 21175 does not define automatic recovery procedures; such procedures, when desired, *shall* be defined at the implementation or application level.

9 Serialization and transport bindings

This clause specifies how the abstract message meta-model and standard message sets defined in Clauses 7 and 8 are bound to concrete encodings and transport mechanisms.

For this edition of ISO 21175-2, a single normative serialization binding based on JSON is defined. Other encodings (for example, XML or binary formats) may be defined in future revisions or in separate technical specifications, provided that they preserve the semantics of the message meta-model.

With respect to transport, this clause provides non-normative guidance for the use of message-oriented middleware and event streaming platforms. The message meta-model and message sets remain independent of any particular product or protocol, provided that the transport service offers asynchronous message delivery with ordering guarantees compatible with Clause 6.

9.1 JSON-based encoding

This subclause defines a normative JSON binding for the message meta-model and standard message sets. JSON encoding shall conform to RFC 8259.

All messages conforming to this part of ISO 21175 *shall* be representable as a single JSON object. That JSON object *shall* contain a minimal set of fields common to all message types and *may* contain additional fields specific to a given message type or implementation profile.

9.1.1 Common JSON structure

Every message *shall* be encoded as a JSON object with at least the fields shown in Table 2. Field names are case-sensitive.

Additional fields may be included, provided that they do not change the meaning of the fields listed in Table 2. Implementations should ignore unknown fields to preserve forward compatibility.

9.1.2 Encoding of primitive types and payloads

Primitive data types defined in 7.4 are mapped to JSON types as follows:

- `boolean` → JSON *true* / *false*;
- `int32`, `int64` → JSON number without a fractional part;
- `float32`, `float64` → JSON number with or without a fractional part;
- `string` → JSON string encoded in UTF-8;
- `bytes` → JSON string containing a Base64 encoding of the octet sequence.

Because JSON numbers are typically implemented using double-precision floating-point representations, the following constraints apply:

- for portable interoperability, implementations *should* restrict integer values to the range that can be represented exactly by an IEEE 754 double-precision number;
- when a wider integer range is required, values *shall* be encoded as JSON strings and their interpretation *shall* be agreed by all participants in the configuration of the simulation run.

Structured payloads and state snapshots are encoded as JSON objects whose properties map to named fields built from these primitive types. Nested objects and JSON arrays may be used as syntactic conveniences, provided that their structure is documented for the relevant message type or port.

Field name	JSON type	Description
"simulationRunId"	string	Identifier of the simulation run to which this message belongs, as defined in 7.2. May be omitted only for errors not associated with any run.
"messageId"	string	Identifier of this message, unique within the scope of the simulation run.
"messageType"	string	Message type identifier. The value shall be one of the strings listed in Table 1.
"senderId"	string	Identifier of the sender within the simulation run (for example, the coordinator identifier, a model identifier for a runner, or another participant).
"receiverId"	string	Identifier of the receiver within the simulation run (for example, the coordinator identifier, a model identifier for a runner, or another participant).
"correlationId"	string (optional)	Identifier used to correlate this message with one or more related messages (for example, a response-request pair).
"eventTime"	number (optional)	Logical event time associated with the message, when applicable, as defined in 7.3.
"nextInternalTime"	number (optional)	Next internal event time reported by the sender, when applicable.
"payload"	object (optional)	Message payload, whose structure depends on "messageType" and, when relevant, on port-level definitions.

Table 2: Common JSON fields for all messages.

9.1.3 Encoding of logical time

Logical time attributes defined in 7.3 are encoded as JSON numbers in the fields "eventTime" and "nextInternalTime".

For a given simulation run:

- all participants (coordinator and runners) *shall* agree on the numeric encoding of logical time (for example, integer or real values);
- logical time infinity *shall* be represented by the maximum finite value of the chosen numeric encoding (for example, the maximum finite `float64` value) and interpreted consistently by all participants;
- wall-clock time, when reported for monitoring or logging, *shall* be encoded using an additional field (for example, "wallClockTime") whose value is an ISO 8601 string; such wall-clock values do not affect simulation semantics.

9.1.4 Port-level payloads

Port-level payloads used in *OutputReport* and *ExecuteTransition* messages are encoded in the "payload" field using the structures defined below.

For an *OutputReport* message, the "payload" object *shall* contain a field "outputs" whose value is a JSON array of port values. Each element of "outputs" *shall* be a JSON object with at least:

- "portName": a string containing the name of an output port of the model;
- "value": a JSON object whose properties are named primitive values as defined in 7.4.

For an *ExecuteTransition* message, the "payload" object *shall* contain a field "inputs" whose value is a JSON array of port values. Each element of "inputs" *shall* be a JSON object with at least:

- "portName": a string containing the name of an input port of the model;
- "value": a JSON object whose properties are named primitive values as defined in 7.4.

All port values in a single message *shall* correspond to the same "eventTime" value of that message.

9.1.5 Message-type specific constraints

Each standard message type defined in Clause 8 constrains the JSON representation as follows.

SimulationInit. A *SimulationInit* message:

- *shall* have "messageType" equal to "SimulationInit";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime";
- *shall* user a "senderId" that is equal to the coordinator's identifier;
- *shall* user a "receiverId" that is equal to the runner's model identifier;

SimulationTerminate. A *SimulationTerminate* message:

- *shall* have "messageType" equal to "SimulationTerminate";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime";
- *may* include in "payload" a field "reason" containing a termination reason code or message.
- *shall* user a "senderId" that is equal to the coordinator's identifier;
- *shall* user a "receiverId" that is equal to the runner's model identifier;

ModelTerminated. A *ModelTerminated* message:

- *shall* have "messageType" equal to "ModelTerminated";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId";
- *shall* user a "senderId" that is equal to the runner's model identifier;
- *shall* user a "receiverId" that is equal to the coordinator's identifier;

NextInternalTimeReport. A *NextInternalTimeReport* message:

- *shall* have "messageType" equal to "NextInternalTimeReport";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime" and "nextInternalTime";
- *shall* user a "senderId" that is equal to the runner's model identifier;
- *shall* user a "receiverId" that is equal to the coordinator's identifier;

RequestOutput / OutputReport. A *RequestOutput* message:

- *shall* have "messageType" equal to "RequestOutput";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime";
- *shall* user a "senderId" that is equal to the coordinator's identifier;
- *shall* user a "receiverId" that is equal to the runner's model identifier;

An *OutputReport* message:

- *shall* have "messageType" equal to "OutputReport";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime";
- *shall* include in "payload" an "outputs" array as defined above;
- *shall* include "nextInternalTime" reporting the next internal time together with outputs;
- *shall* user a "senderId" that is equal to the runner's model identifier;
- *shall* user a "receiverId" that is equal to the coordinator's identifier;

ExecuteTransition / TransitionComplete. An *ExecuteTransition* message:

- *shall* have "messageType" equal to "ExecuteTransition";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime";
- *shall* include in "payload":
 - * optionally, an "inputs" array as defined above.
- *shall* user a "senderId" that is equal to the coordinator's identifier;
- *shall* user a "receiverId" that is equal to the runner's model identifier;

A *TransitionComplete* message:

- *shall* have "messageType" equal to "TransitionComplete";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId", "eventTime" and "nextInternalTime";
- *shall* user a "senderId" that is equal to the runner's model identifier;
- *shall* user a "receiverId" that is equal to the coordinator's identifier;

MonitoringMessage. A *MonitoringMessage*:

- *shall* have "messageType" equal to "MonitoringMessage";
- *shall* contain "simulationRunId", "messageId", "senderId", "receiverId";
- *may* contain "eventTime" when the observation is associated with a simulation event;
- *shall* include in "payload" at least:
 - * a field "category" (string) indicating the nature of the observation (for example, "stateSnapshot", "metric", "trace", "debug");
 - * a field "values" which is a JSON object containing primitives types defined above;
 - * optionally, fields "sourceRole" implementing the source role identifier defined in Clause 8.

ErrorReport. An *ErrorReport* message:

- *shall* have "messageType" equal to "ErrorReport";
- *shall* contain "messageId", "senderId", and "receiverId";
- *shall* contain "simulationRunId" when the error is associated with a specific simulation run. Must be set to an empty string if no simulation can be linked to the error.
- *shall* include in "payload" at least:
 - * "originRole": a string with value "Coordinator", "Runner" or "Other", implementing the origin role identifier of Clause 8;
 - * "originId": a string identifying the component that detected the error;
 - * "severity": a string with value "info", "warning", "error" or "fatal";
 - * "errorCode": an integer identifying the category of error;
 - * "message": a string containing a human-readable description.
- *may* include additional fields in "payload" for diagnostic details.

In most cases, "originId" will be equal to "senderId", but they can be different, for example when an intermediary component reports an error detected in another participant.

9.1.6 Illustrative example

The JSON object below illustrates an *OutputReport* message produced by a runner:

```
{
  "simulationRunId": "run-1234",
  "messageId": "msg-00042",
  "messageType": "OutputReport",
  "senderId": "model-CLERK-1",
  "receiverId": "coordinator-STORE-1",
  "eventTime": 2.0,
  "nextInternalTime": 2.0,
  "payload": {
    "outputs": [
      {
        "portName": "depart",
```

```

    "value": {
      "twait": 1.0,
      "tenter": 1.0,
      "tleave": 2.0
    }
  }
]
}
}

```

This example is informative only; any JSON instance that satisfies the rules of this subclause and the semantics of Clauses 7 and 8 is conformant.

9.2 Transport protocol considerations

This subclause provides non-normative guidance for mapping the message meta-model and standard message sets onto message-oriented middleware or event streaming platforms. It does not mandate the use of a specific product, protocol or infrastructure technology.

9.2.1 General principles

Regardless of the chosen transport technology, the following principles apply:

- messages exchanged during a given simulation run should be delivered in an order consistent with the ordering constraints defined in Clauses 6 and 8;
- the transport service should preserve the contents of messages except for transformations required by the chosen serialization;
- the transport service should provide a delivery behaviour compatible with reliable collaborative execution; when duplicate delivery may occur, implementations may detect and handle duplicates using the "messageId" and "correlationId" fields.

9.2.2 Shared logical channel organization

For a given simulation run, protocol messages are exchanged through a single shared logical channel, as described in Clause 6.

In such a mapping:

- all participants in the simulation run publish messages to the same shared logical channel;
- all participants in the simulation run consume messages from that same shared logical channel;
- each participant uses the "receiverId" field to determine whether a received message is intended for it;
- messages whose "receiverId" does not match the identifier of the receiving participant are ignored.

The concrete realization of this shared logical channel is an implementation concern. Depending on the chosen infrastructure, it may be mapped to a topic, queue, stream or equivalent communication construct, provided that the ordering and addressing requirements defined in this document are preserved.

9.2.3 Delivery and implementation considerations

Implementations should ensure that the chosen transport mapping remains compatible with the coordinator–runner interaction semantics defined in this document.

In particular:

- the transport configuration should allow all participants in a simulation run to observe the shared logical channel consistently;
- the mapping between the abstract shared logical channel and concrete middleware constructs is outside the scope of this part of ISO 21175;
- infrastructure-specific concerns such as channel creation, endpoint naming, deployment, access control, persistence and broker configuration are outside the scope of this part of ISO 21175 and may be addressed in other parts of ISO 21175 or in implementation-specific documentation.

9.2.4 Ordering and partitions

When the transport platform partitions topics into independent, ordered sub-streams (for example, partitions in Apache Kafka), the following constraints apply:

- all messages for a given topic used in a simulation run *shall* be delivered to consumers in a total order compatible with the ordering constraints of Clauses 6 and 8;
- when this guarantee is provided per partition, implementations *shall* ensure that all messages sent on a given topic for a given simulation run are written to a single partition; in platforms where this is controlled by a message key, a consistent keying strategy *shall* be used;
- it is recommended, for simplicity, that topics used by a simulation run be configured with a single partition when possible, to obtain a straightforward mapping between the abstract channels of Clause 6 and the concrete transport.

9.2.5 Delivery guarantees and recovery

This part of ISO 21175 does not mandate a specific choice of delivery guarantees (for example, at-most-once, at-least-once or exactly-once). However:

- at-least-once delivery is recommended for messages that affect simulation semantics; implementations detecting duplicates via "messageId" or "correlationId" *shall* handle them in a manner that preserves the semantics of the underlying formalism;
- monitoring and logging messages may adopt weaker guarantees, subject to the needs of the application.

Recovery procedures after transport-level failures (for example, replay from persistent logs, restoring state from checkpoints) are out of scope of this part and may be addressed at the implementation level or in other parts of ISO 21175 dealing with service and API specifications.

9.2.6 Security and isolation

Security mechanisms (authentication, authorization, confidentiality and integrity protection) depend on the chosen transport technology and the policies of the collaborating organizations. This part of ISO 21175 does not mandate specific mechanisms, but implementers should ensure that:

- only authorized participants can publish to or subscribe from the topics or channels used by a simulation run;
- data in transit can be protected against unauthorized disclosure and modification, when required by the application domain.

Details such as network addressing, service endpoints and low-level security configuration are out of scope of this part and may be handled in other parts of ISO 21175 or in implementation-specific documentation.

10 Extensibility and future work

This clause will describe how the message meta-model and message sets can be extended while preserving interoperability:

- mechanisms for adding new message types and payload fields;
- versioning strategy for the meta-model and encodings;
- guidelines for domain-specific extensions agreed by subgroups of users.

Annex A (informative)

Example message sequences for a simple collaborative simulation

Two DEVS-based frameworks initiated a simple collaborative simulation of a single clerk store. A generator model generates a stream of customers, each with an arrival time and a waiting time, the time required for checkout -based on their shopping. A clerk model manages the checkout line, processing each customer according to their waiting time. When a customer is complete, it generates a customer output, marking the customer with its departure time. An observer receives each departing customer and computes maximum and average waiting times. This example is based on the Adevs BusyMart example. For this example, a CMSE was established with a coordinator using the DEVS Streaming Framework and a runner using Adevs wrapped with a library to implement the standard message set and behavior. In the CMSE, Adevs implemented the clerk model and a runner to control its execution. The DEVS Streaming Framework implemented the generator, the observer, and the coordinator to handle messages to and from the runner. Apache Kafka implemented the streaming platform using a shared topic, called "BusyMartSimulation-Store". The "storeCoordinator" coordinator and the "clerk1" runner shared this topic for input and output. The simulation progressed as follows.

The coordinator initialized the simulation, creating the local generator and observer. It then sent the following SimulationInit message to the runner. It initializes the simulation start time of 0.0 and simulation id of "BusyMartSimulation". Note that the CMSE has agreed to use 64 bit floating point types to represent time. CMSE runners will have to ignore unknown fields in their JSON processing.

```
{
  "messageType": "SimulationInit",
  "eventTime": 0.0,
  "simulationRunId": "BusyMartSimulation",
  "messageId": "3a802973-545f-4efe-a3e0-9db82d7cc51d",
  "senderId": "storeCoordinator",
  "receiverId": "clerk1"
}
```

Upon receipt, the runner initializes clerk1 as empty and idle with an initial time of 0.0. Its next internal time is infinity, represented as the maximum float64 value in the following NextInternalTimeReport.

```
{
  "eventTime": 0.0,
  "messageId": "d28f9529-b287-43bc-9a2f-be1f811f052e",
  "messageType": "NextInternalTimeReport",
  "nextInternalTime": 1.7976931348623157e+308,
  "receiverId": "storeCoordinator",
  "senderId": "clerk1",
  "simulationRunId": "BusyMartSimulation"
}
```

At $t=1.0$, the first customer arrives from the generator. The coordinator sends the following ExecuteTransition message to the runner to initiate an external state transition. Note the receiverId set to "clerk1" and an inputs array containing the first customer, with an arrival time of 1.0 and wait time of 1.0, to the "arrive" port. Also note the "@class" field in the value field added by the DEVS Streaming Framework serialization engine. Upon CMSE agreement, the runner will use this field to identify the data type and deserialize it in its own framework to a C++ Customer.

```
{
  "messageType": "ExecuteTransition",
  "eventTime": 1.0,
  "payload": {
    "@class": "devs.iso.ExecuteTransitionPayload",
    "inputs": [
      {
        "value": {
          "@class": "cloud.simlytics.devssfstore.Customer",
          "twait": 1.0,
          "tenter": 1.0,
          "tleave": 0.0
        },
        "portName": "arrive"
      }
    ]
  },
  "simulationRunId": "BusyMartSimulation",
  "messageId": "1c3e3183-8e2f-4c04-8ada-ae547365f4e2",
  "senderId": "storeCoordinator",
}
```

```
"receiverId": "clerk1"
}
```

The runner receives the ExecuteTransition message, calls the external transition function of the clerk1 model. This adds the first customer to the queue. The time advance function returns a value of 2.0, when the customer will depart the store. This generates the following TransitionComplete message:

```
{
  "eventTime": 1.0,
  "messageId": "9ef1c43b-5f49-445d-826d-a5b35f1dad44",
  "messageType": "TransitionComplete",
  "nextInternalTime": 2.0,
  "receiverId": "storeCoordinator",
  "senderId": "clerk1",
  "simulationRunId": "BusyMartSimulation"
}
```

The coordinator uses the "nextInternalTime" field of the TransitionComplete message to compare it to scheduled transitions in the generator and observer. Also at t=2.0, the generator executes an internal transition and generates an new customer arrival, which will be routed to the clerk. This corresponds to a DEVS confluent state transition. Prior to transition, the coordinator sends the following RequestOutput to the runner.

```
{
  "messageType": "RequestOutput",
  "eventTime": 2.0,
  "simulationRunId": "BusyMartSimulation",
  "messageId": "79d1c6d4-29e8-4472-8e5f-b113cd6f96d5",
  "senderId": "storeCoordinator",
  "receiverId": "clerk1"
}
```

Upon receipt, the runner executes the output function of the clerk model and generates the following OutputReport, indicating a customer finishing checkout at t=2.0 and leaving via the "depart" port. In accordance with a CMSE agreement, the data type of the customer is indicated with the "@class" field in the "outputs.value" object.

```
{
  "eventTime": 2.0,
  "messageId": "e2d0d42b-35a2-4f81-a488-5a6d9933ffc9",
  "messageType": "OutputReport",
  "nextInternalTime": 2.0,
  "payload": {
    "@class": "devs.iso.OutputReportPayload",
    "outputs": [
      {
        "portName": "depart",
        "value": {
          "@class": "cloud.simlytics.devssfstore.Customer",
          "tenter": 1.0,

```

```

        "tleave": 2.0,
        "twait": 1.0
    }
}
],
},
"receiverId": "storeCoordinator",
"senderId": "clerk1",
"simulationRunId": "BusyMartSimulation"
}

```

The coordinator processes the `OutputReport` and sends the customer to the observer model. This generates no other messages, so the coordinator then sends the following `ExecuteTransition` message to the runner at $t=2.0$.

```

{
  "messageType": "ExecuteTransition",
  "eventTime": 2.0,
  "payload": {
    "@class": "devs.iso.ExecuteTransitionPayload",
    "inputs": [
      {
        "value": {
          "@class": "cloud.simlytics.devssfstore.Customer",
          "twait": 4.0,
          "tenter": 2.0,
          "tleave": 0.0
        },
        "portName": "arrive"
      }
    ]
  },
  "simulationRunId": "BusyMartSimulation",
  "messageId": "c9321687-a4f6-4102-ba2e-ca68c06ee549",
  "senderId": "storeCoordinator",
  "receiverId": "clerk1"
}

```

The runner receives the `ExecuteTransition` message and calls the confluent transition function of the `clerk1` model. This adds the new customer to the queue. The time advance function returns a value of 6.0, when this customer will depart the store. This generates the following `TransitionComplete` message:

```

{
  "eventTime": 2.0,
  "messageId": "4ae987ba-ebc7-474f-a680-86927b53a7c8",
  "messageType": "TransitionComplete",
  "nextInternalTime": 6.0,
  "receiverId": "storeCoordinator",
  "senderId": "clerk1",
  "simulationRunId": "BusyMartSimulation"
}

```

The BusyMartSimulation continues to execute with the clerk1 processing customers from the generator until we reach $t=45$, when there are no more customers coming from the generator, and the clerk's queue is empty. This is indicated by the next internal time of the runner and all other models equal to infinity. In order to stop the simulation, the coordinator sends the following SimulationTerminate message to the runner.

```
{
  "messageType": "SimulationTerminate",
  "eventTime": 45.0,
  "simulationRunId": "BusyMartSimulation",
  "messageId": "bb0669cd-05d3-4767-bc12-a2af5a2fadf3",
  "senderId": "storeCoordinator",
  "receiverId": "clerk1",
  "payload": {
    "@class": "devs.iso.SimulationTerminatePayload",
    "reason": "Simulation completed"
  }
}
```

Upon receipt, the runner shuts down the clerk1 model and responds with the following ModelTerminated message.

```
{
  "messageId": "d4b26705-d14b-4331-862f-bd1c677ed9e1",
  "messageType": "ModelTerminated",
  "receiverId": "storeCoordinator",
  "senderId": "clerk1",
  "simulationRunId": "BusyMartSimulation"
}
```

Upon receipt, the coordinator shuts down the generator and observer models then stops the messaging infrastructure. Note that the ModelTerminated message is an optional message. The coordinator could shut down immediately after sending the SimulationTerminate message. However, this CMSE uses shared infrastructure for logging, so waiting for a ModelTerminated message ensures the infrastructure won't be shut down before the clerk model is done using it.

References

- [1] ISO 21175-1, *Automation systems and integration — Collaboration Environment Requirements of Simulation on Different Manufacturing Platforms — Part 1: Reference Model and Process*.
- [2] Zeigler, B.P., *Multifaceted Modelling and Discrete Event Simulation*. Academic Press, 1984.
- [3] Prähofer, H., "System theoretic formalisms for combined discrete-continuous system simulation," *International Journal of General Systems*, vol. 19, no. 3, pp. 219–240, 1991.
- [4] Kim, Y.J.; Kim, T.G., "A heterogeneous distributed simulation framework based on DEVS formalism," in *Proceedings of the Sixth Annual Conference on Artificial Intelligence, Simulation and Planning in High Autonomy Systems*, pp. 116–121, 1996.

- [5] Wang, Y.; Zeigler, B.P., “Implementing the DEVS formalism on a parallel SIMD architecture,” *International Journal of Computer Simulation*, vol. 5, pp. 229–245, 1995.
- [6] Zeigler, B.P.; Muzy, A.; Kofman, E., *Theory of Modeling and Simulation: Discrete Event and Iterative System Computational Foundations*, 3rd ed. Academic Press, 2018.
- [7] Zeigler, B.P.; Kewley, R.; Wainer, G., “DEVS Closure Under Coupling, Universality, and Uniqueness: Enabling Simulation and Software Interoperability from a System-Theoretic Foundation,” *Computers*, vol. 14, no. 12, art. 514, 2025.

TODO: add more references